Software Design Description for Drift

4072 Senior Project 2

August Wetterau

Contents

Software Design Desciption **2**

6 Software Design Description **3**
6.1 Introduction Section . . . 3
6.1.1 System Objective . . . 3
6.1.2 Hardware, Software, and Human Interfaces Section . . . 3
6.1.2.1 User Interface . . . 3
6.1.2.2 Mobile Device Hardware . . . 3
6.1.2.3 Microphone Input . . . 3
6.1.2.4 Speaker Output . . . 3
6.1.2.5 Hume AI API . . . 4
6.1.2.6 Supabase Service . . . 4
6.2 Architectural Design Section . . . 4
6.2.1 Major Software Components Section . . . 4
6.2.2 Major Software Interactions Section . . . 4
6.2.3 Architectural Design Diagrams Section . . . 5
6.3 Detailed CSC and CSU Descriptions Section . . . 6
6.3.1 Detailed Class Descriptions Section . . . 6
6.3.1.1 User Management Class File . . . 6
6.3.1.2 Scenario Selection View . . . 7
6.3.1.3 Conversation View . . . 7
6.3.1.4 Feedback & Scoring View . . . 7
6.3.1.5 Progress Tracking View . . . 7
6.3.1.6 Conversation Session Class File . . . 8
6.3.1.7 Hume AI Service . . . 8
6.3.1.8 Supabase Service . . . 8
6.3.2 Detailed Interface Descriptions Section . . . 8
6.3.2.1 User Management Class File . . . 8
6.3.2.2 Scenario Selection View . . . 9
6.3.2.3 Conversation View . . . 9
6.3.2.4 Feedback & Scoring View . . . 9
6.3.2.5 Progress Tracking View . . . 9
6.3.2.6 Conversation Session Class File . . . 9
6.3.2.7 Hume AI Service . . . 9
6.3.2.8 Supabase Service . . . 9
6.3.3 Detailed Data Structure Descriptions Section . . . 9
6.3.3.1 User Management Class File . . . 10
6.3.3.2 Scenario Selection View . . . 10
6.3.3.3 Conversation View . . . 10
6.3.3.4 Feedback & Scoring View . . . 10
6.3.3.5 Progress Tracking View . . . 10
6.3.3.6 Conversation Session Class File . . . 10
6.3.3.7 Hume AI Service . . . 10
6.3.3.8 Supabase Service . . . 10
6.3.4 Detailed Design Diagrams . . . 11

6.4 Database Design and Description Section 13
6.4.1 Database Design ER Diagram Section 13
6.4.2 Database Access Section 14
6.4.3 Database Security Section 14

List of Figures

1 Use Case Diagram 5
2 Class Diagram 5
3 Deployment Diagram 6
4 Class Diagram 11
5 Application Activity Diagram 12
6 Sequence Diagram 13
7 Database Diagram 14

6 Software Design Description

6.1 Introduction Section

This document presents the architecture and detailed design for the software of the Drift application. Drift is an app that uses conversational Al to put users into professional situations to improve their speech. After every scenario, the transcript will be sent to an external Al service, which will score the user's responses through text, and vocal tone. It will give specific feedback to help the user understand what to focus on improving. To do this, the system must be composed of multiple components that work together to deliver the application functionality. The application will use Hume AI for conversation and scoring, and Supabase for data storage. It will have a user interface that allows the users to select scenarios, practice conversations, and view results. It will also process the user's speech through the microphone, and receive responses. After each conversation, the user will receive feedback on what they did right and wrong with explicit details on why each of these were right or wrong. Finally, a data storage component is required so the user can look at their previous scores and past sessions to track improvement. This will all communicate over the internet for a real time conversation feel.

6.1.1 System Objective

The objective of this application is to help users improve communication skills, specifically in professional settings. Real professional scenarios are simulated so the user can put themselves in a situation and see how they would react, such as meeting someone at a career fair. The application uses AI to analyze the transcript and the vocal tone to provide meaningful feedback and specific areas of improvement. The user can also track improvements over time.

6.1.2 Hardware, Software, and Human Interfaces Section

The following subsections describe the hardware, software, and human interfaces used in the Drift application.

6.1.2.1 User Interface The user will interact with the application through clear buttons and visual cues on the screen. There will be tabs for scenario selection, conversation history, and the user profile. From the scenario selection, the user will be able to click a scenario, bringing them to the conversation view, and after ending, to the feedback page. The conversation history will have a list of past conversations, and clicking on one will bring the user to the feedback page for that session.

6.1.2.2 Mobile Device Hardware The application will run on iOS and Android phones but will be limited to mobile devices. All modern iOS and Android phones should be able to run this application, as most of the computation is done externally. The mobile device will be used to capture user input, display the interface, and connect to external services over the internet. Older mobile devices may not be able to run the application if they struggle with many other modern applications.

6.1.2.3 Microphone Input A microphone attached to the device is required to capture the user's speech for conversations. Most modern mobile devices have built in microphones, so they would not need external hardware.

6.1.2.4 Speaker Output A speaker or headphones attached to the user's device is required to play the AI generated responses. Most modern mobile devices have this built in, or allow the user to connect to external devices.

6.1.2.5 Hume AI API The application will connect with Hume AI to use its real time conversation capabilities. It will also use this AI to score the user's conversation after it has ended. It will use the internet to communicate with this service, meaning the device will need internet access.

6.1.2.6 Supabase Service The application will use Supabase for the database and authentication. It will store basic user data, session data, and scenario data.

6.2 Architectural Design Section

The high-level structure of my project is mainly built around the mobile frontend, Supabase backend, and an external HUME conversational AI component. This system is designed with modularity, meaning each subsystem can operate independently, but is also connected through APIs. The frontend includes a home page, scenario selection, conversation view, conversation analysis screen, and a profile as its views. The backend has multiple parts, but consists of a user table, scenario table, and an authentication service with security rules. The AI will be an external tool, connected to the app and called by an API key. This will be the HUME EVI 3 which has a conversational AI mode for speech to speech conversations.

6.2.1 Major Software Components Section

Frontend (Mobile App)

- React Native framework
- Component-based architecture
- Handles any user interactions such as scenario selection, viewing history, starting conversation, displaying score, and seeing profile

Backend (Supabase)

- Authentication with Supabase Auth
- Database with Supabase Postgres for user profiles, scenarios, and score saving
- Edge Functions for secure data processing

AI using External Integration

- API: HUME EVI 3
- Secure OAuth connection

6.2.2 Major Software Interactions Section

The frontend and backend of my app will use RESTful APIs, HTTPS, and JSON data exchange to communicate with one another. I will use tables like "users" and "scenarios" to store information in the backend that will be used by the frontend. Supabase SDK handles all the Authentication, user session, and data retrieval. The only part on the frontend that needs to be done is to save a token to the user's phone when they are logged in, so they do not need to log in every time they open the app. These tokens will be encrypted so only the user can access them. The backend will communicate with the external API (HUME AI) through an OAuth token. The users will communicate with the application using the UI flow that I designed. The app will communicate with the backend to call the HUME API to start a conversation, which will be sent to the frontend to display, speak to the user using the built in speaker. It will also send the user's microphone input to the backend, which will send it to the HUME AI for a response.

6.2.3 Architectural Design Diagrams Section

The following section contains the system diagrams for the Drift App:

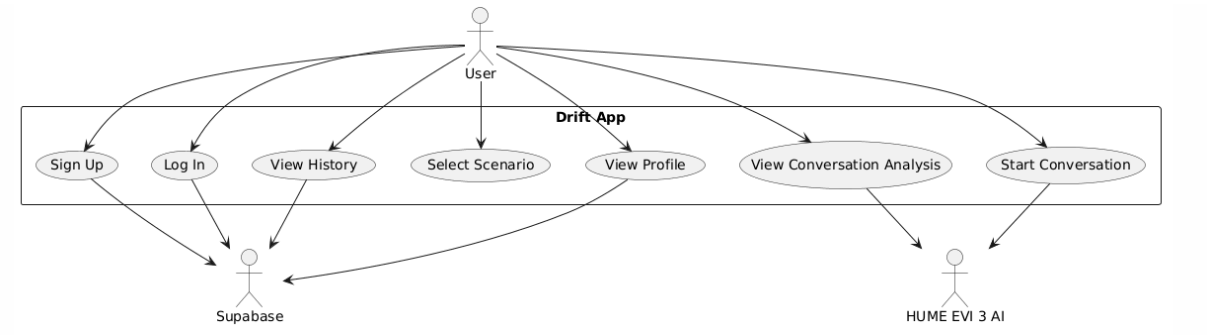

Figure 1: Use Case Diagram

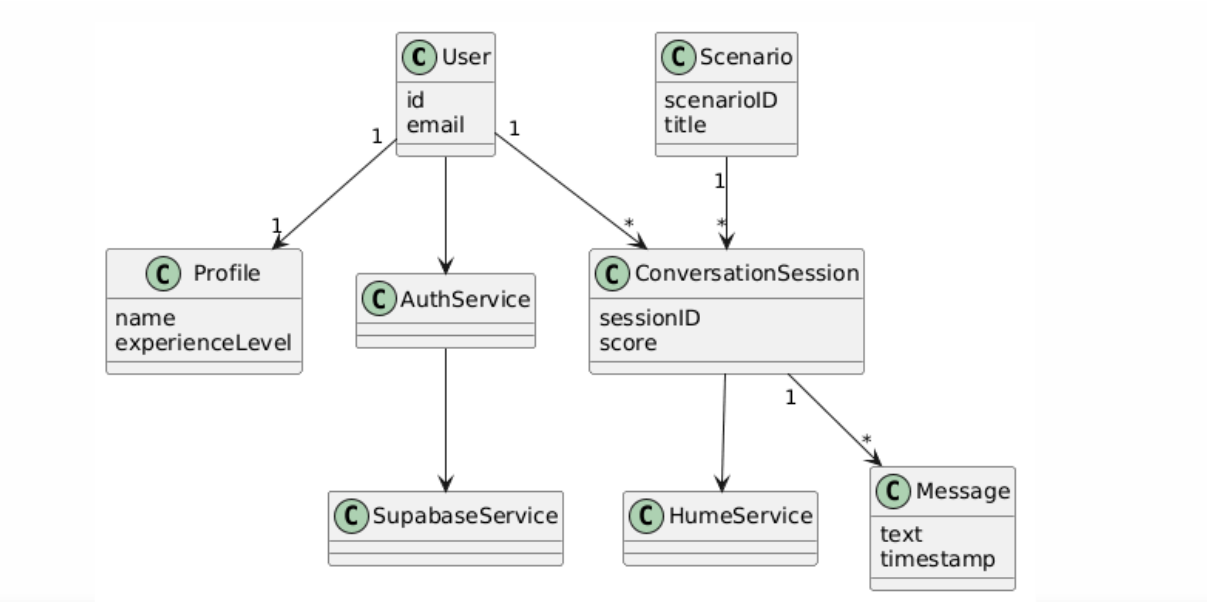

Figure 2: Class Diagram

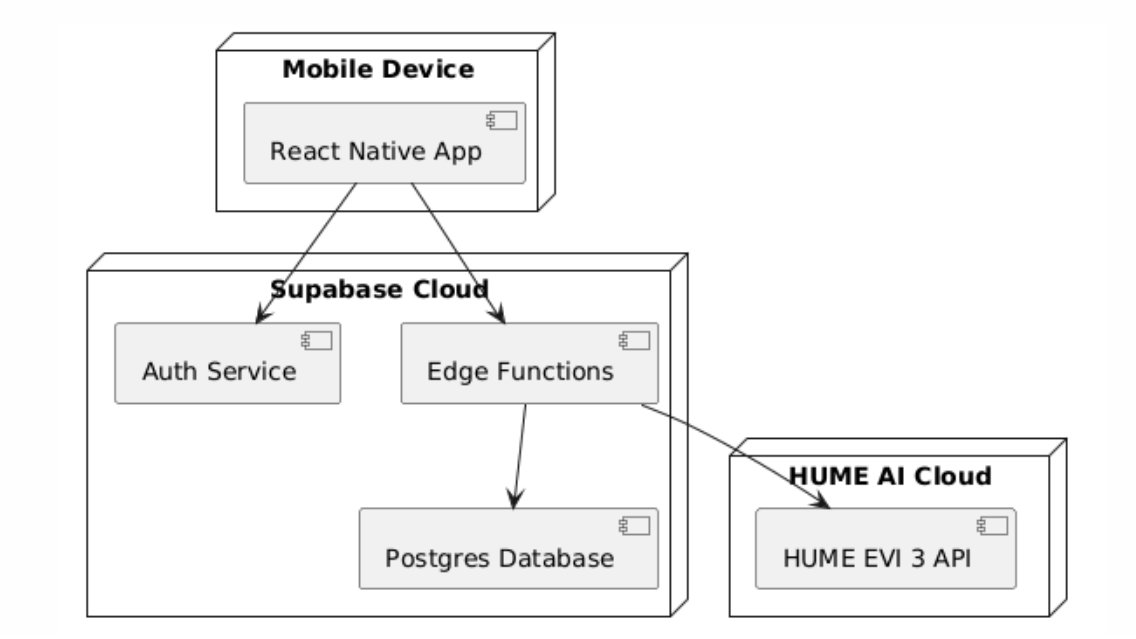

Figure 3: Deployment Diagram

6.3 Detailed CSC and CSU Descriptions Section

The *Drift* application contains the following Computer Software Components (CSC), each composed of the following Computer Software Units (CSU):

- Authentication Service
  - User Management Class File
- View Models
  - Scenario View
  - Conversation View
  - Feedback View
  - Progress View
- Backend Services
  - Supabase Service
  - Conversation Session Class File
- AI Integration Module
  - Hume AI Service

6.3.1 Detailed Class Descriptions Section

The following section provides the details of all classes used in the *Drift* application.

6.3.1.1 User Management Class File

The user management class handles user authentication, session management, and user identity. It contains the following classes:

UserAuth:

- Fields:
  - AuthToken: This allows a user to stay logged in on a device by storing a token on the phone.
- Methods:
  - LogIn: Uses the user's credentials to log them into the app.

- SignUp: Creates a new AuthToken and registers their information into the database.
- GetCurrentUser: Returns the AuthToken for the current user.
- LogOut: Logs the user out, removing their AuthToken from the device.

6.3.1.2 Scenario Selection View

This view allows users to look at and select different conversation scenarios

ScenarioView:

- Fields:
  - ScenarioList: This dictionary contains every scenario id and its name and description.
  - SelectedScenario: The selected scenario id string.
- Methods:
  - FetchScenarios: Retrieves all the scenario names, descriptions, and ids from the database.
  - SelectScenario: Sets the selected scenario to the chosen id
  - DisplayScenarioDetails: Uses the scenario id to return the name and description from the ScenarioList dictionary.

6.3.1.3 Conversation View

This view manages real time conversation between user and AI.

ConversationView:

- Fields:
  - CurrentSession: Stores the session id, selected scenario, and the time it started.
  - AudioInput: Captures the users audio to send to the AI.
- Methods:
  - StartSession: Starts a new conversation session through Hume, returning the session id.
  - SendAudio: Sends the Audio to Hume AI for processing
  - ReceiveResponse: Receives AI audio to play out loud.
  - EndSession: Ends the current session.

6.3.1.4 Feedback & Scoring View

This view shows scores and feedback after a conversation.

FeedbackView:

- Fields:
  - Score: Total score from the AI.
  - FeedbackText: Stores the feedback generated by the AI including strengths and weaknesses.
- Methods:
  - FetchFeedback: Retrieves score and feedback data from the AI.
  - DisplayScore: Shows the total score to the user.
  - DisplayFeedback: Shows the detailed feedback to the user.

6.3.1.5 Progress Tracking View

This view allows users to see previous sessions.

ProgressView:

- Fields:
  - SessionHistory: Stores past conversation scenario and date.
  - ScoreHistory: Stores scores from previous sessions.
- Methods:

- FetchHistory: Retrieves past sessions and scores from the database.
- DisplayHistory: Shows list of sessions and scores.

6.3.1.6 Conversation Session Class File

This class represents a conversation and its data.

ConversationSession:

- Fields:
  - SessionID: Unique ID assigned to each session
  - Transcript: Stores a list of the ordered messages between the user and AI.
  - Score: Stores the final score generated after the session.
- Methods:
  - SaveSession: Saves session data such as the id transcript and score to the database.
  - LoadSession: Retrieves session data from the database using the id.

6.3.1.7 Hume AI Service

This class handles communication with the Hume AI API.

HumeService:

- Fields:
  - APIToken: The API key to authenticate requests to the Hume API.
  - EndpointURL: The API endpoint for sending requests.
- Methods:
  - SendAudio: Sends user audio to the API for processing.
  - ReceiveResponse: Returns the AI response from the API.
  - AnalyzeConversation: Sends all conversation data to AI for scoring and feedback.

6.3.1.8 Supabase Service

This class handles the communication between the application and the database.

SupabaseService:

- Fields:
  - DatabaseClient: Manages the connection to the database
- Methods:
  - SaveUser: Stores the user data in the database.
  - GetUser: Returns the user data from the database
  - SaveSession: Stores the session data in the database.
  - GetSessions: Returns the session data from the database.

6.3.2 Detailed Interface Descriptions Section

This section describes the interfaces for each of the computer software units in the *Drift* application.

6.3.2.1 User Management Class File

UserAuth:

- Interfaces:
  - Supabase Database: Reads and writes user data to the database.
  - Supabase Authentication: Handles user login, signup, and sessions.

6.3.2.2 Scenario Selection View

ScenarioView:

- Interfaces:
  - Database: Returns scenario data from the database.

6.3.2.3 Conversation View

ConversationView:

- Interfaces:
  - Hume AI API: Sends audio input and receives AI audio response.
  - Microphone Input: Uses built in microphone from device to capture audio.
  - Speaker Output: Plays AI audio response to the user

6.3.2.4 Feedback & Scoring View

FeedbackView:

- Interfaces:
  - Hume AI API: Returns scores and feedback after a session.
  - Database: Accesses scores and feedback data.

6.3.2.5 Progress Tracking View

ProgressView:

- Interfaces:
  - Database: Returns past sessions and scores for the user.

6.3.2.6 Conversation Session Class File

ConversationSession:

- Interfaces:
  - Database: Stores and retrieves session data.

6.3.2.7 Hume AI Service

HumeService:

- Interfaces:
  - Hume AI API: Handles sending audio and receiving AI response.
  - OAuth Authentication: Manages secure access to Hume AI using API Tokens.

6.3.2.8 Supabase Service

SupabaseService:

- Interfaces:
  - Supabase Database: Reads and writes user and session data.
  - Supabase Authentication: Manages user login, signup, and session authentication.

6.3.3 Detailed Data Structure Descriptions Section

This section describes the data structures for each of the computer software units in the *Drift* application.

6.3.3.1 User Management Class File

UserAuth:

- Data Structures:
  - None: Authentication data is handled externally by Supabase and not stored locally.

6.3.3.2 Scenario Selection View

ScenarioView:

- Data Structures:
  - Dictionary: Stores scenarios in a dictionary with the id being the key and the name and description being the values.

6.3.3.3 Conversation View

ConversationView:

- Data Structures:
  - None: Data is streamed during a conversation and not stored in this view.

6.3.3.4 Feedback & Scoring View

FeedbackView:

- Data Structures:
  - Object: Stores scores and feedback data.

6.3.3.5 Progress Tracking View

ProgressView:

- Data Structures:
  - ArrayList: Stores the list of past conversation sessions.
  - ArrayList: Stores the list of past scores.

6.3.3.6 Conversation Session Class File

ConversationSession:

- Data Structures:
  - Object: Stores session data such as session id, transcript, and score.

6.3.3.7 Hume AI Service

HumeService:

- Data Structures:
  - None: Does not store any audio data locally.

6.3.3.8 Supabase Service

SupabaseService:

- Data Structures:
  - Database Tables: Stores user and session data in tables.

6.3.4 Detailed Design Diagrams

The following section details the diagrams for the *Drift* Application.

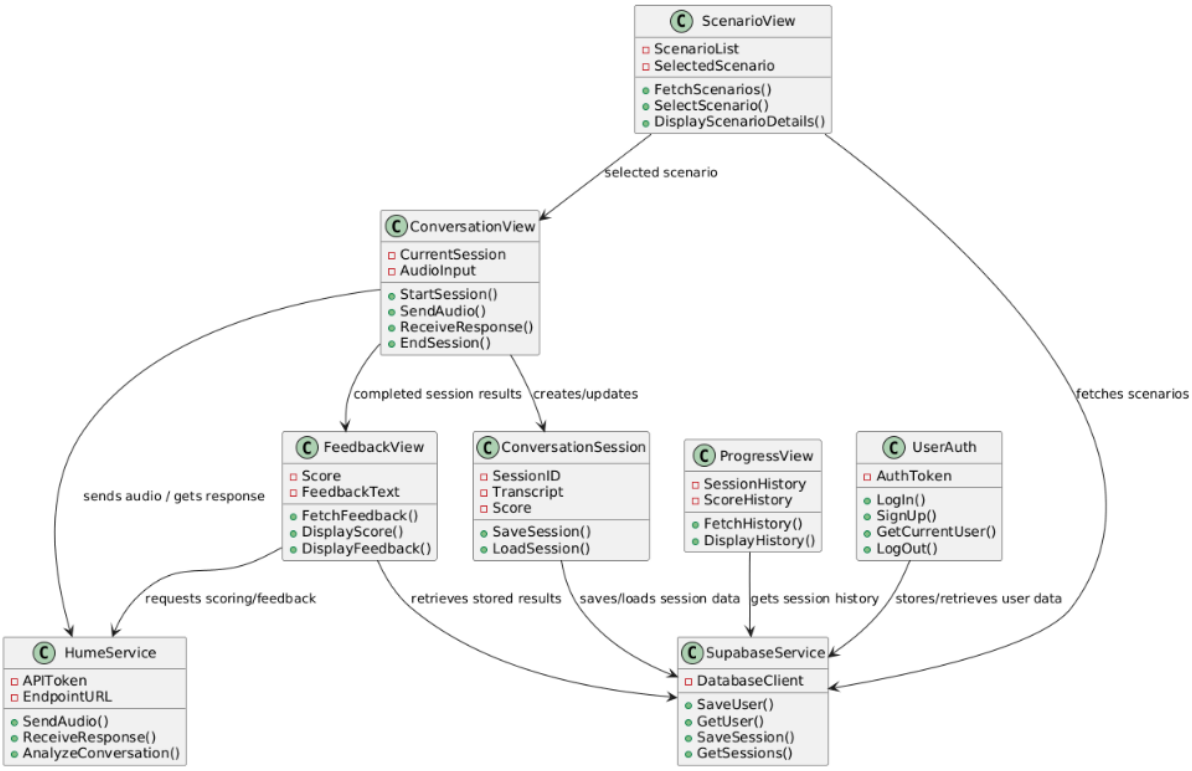

Figure 4: Class Diagram

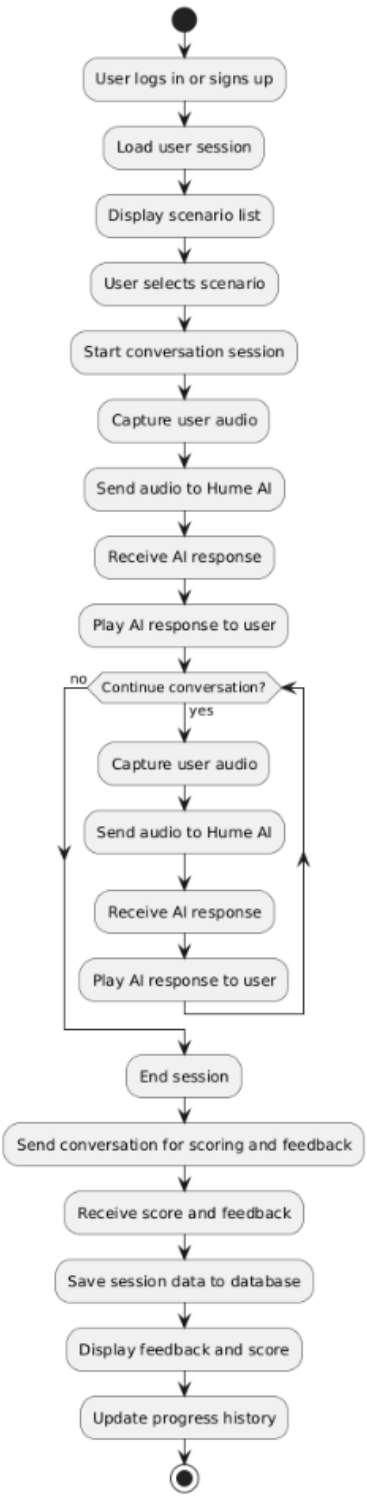

Figure 5: Application Activity Diagram

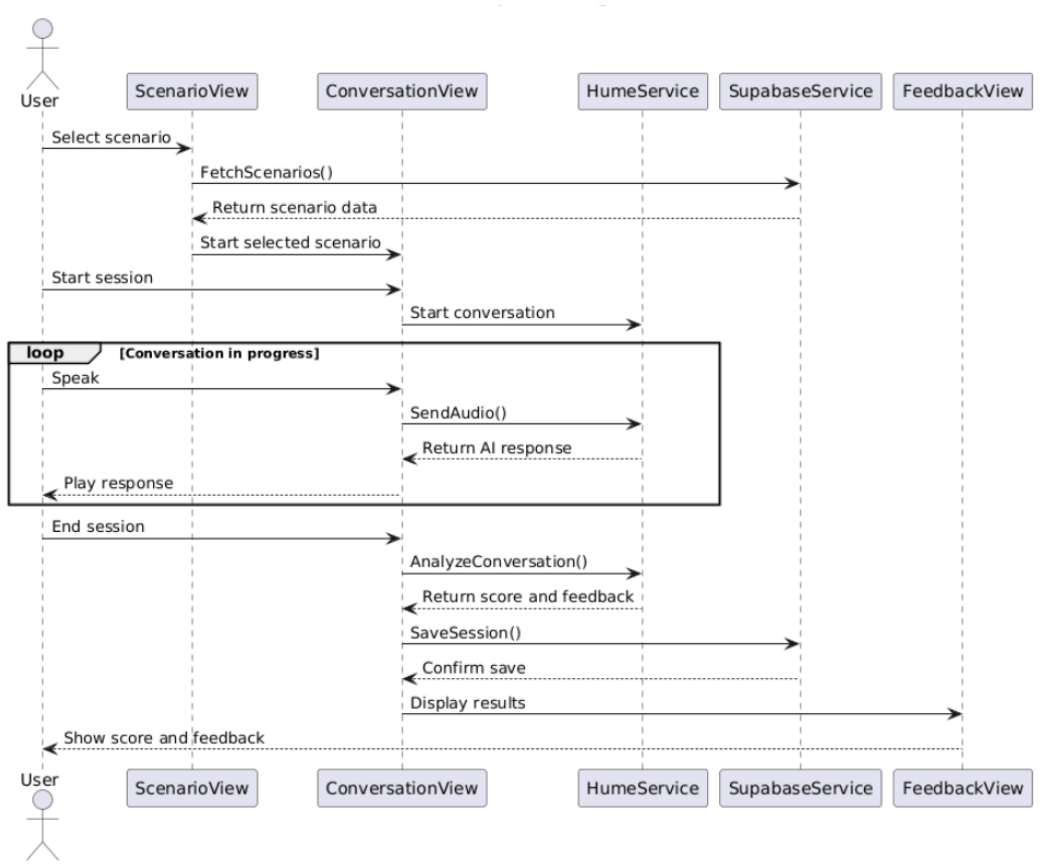

Figure 6: Sequence Diagram

6.4 Database Design and Description Section

This section describes the database used in the *Drift* application.

6.4.1 Database Design ER Diagram Section

Figure 7 shows the database diagram for the *Drift* application.

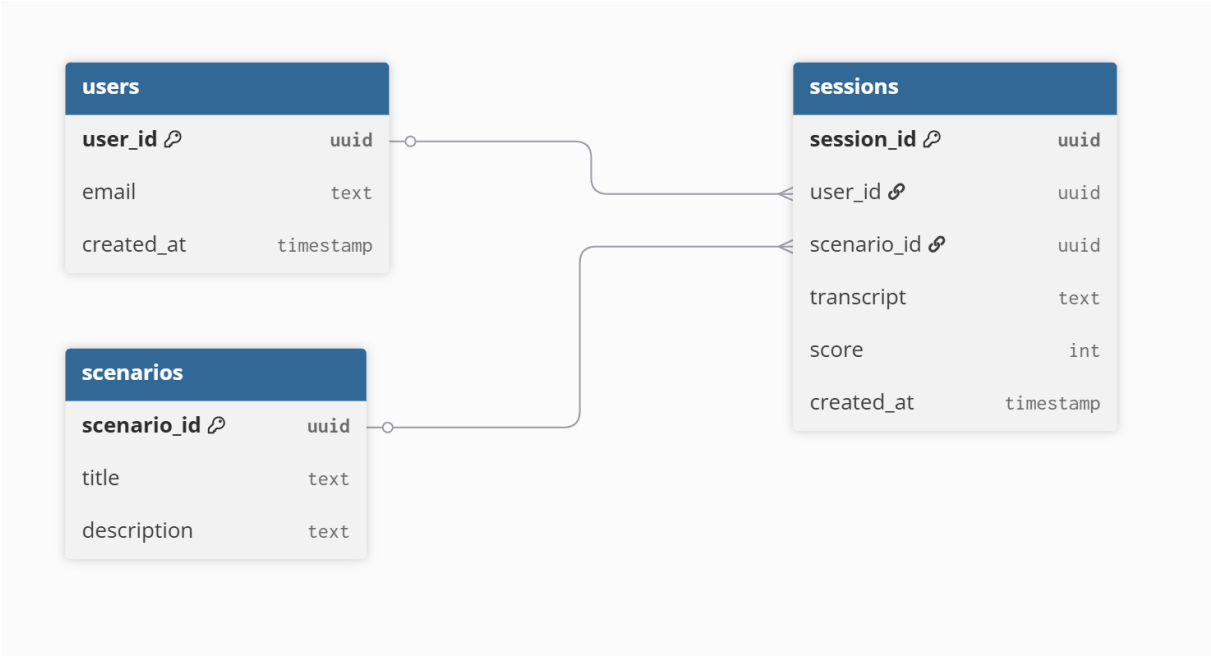

Figure 7: Database Diagram

6.4.2 Database Access Section

The Supabase SDK will be used to access the database. It has functions built in for reading, writing, updating, and deleting tables in a very easy way. It lets us query specific rows in the tables, and has filtering/ordering support. There is very minimal code needed to perform these functions and it is very easy to implement them all. For this application, only very simple tasks will be done. The AI will be used for conversations and scoring, while the database will be used to store and retrieve application data. Scenario data will be accessed from the database when the user accesses the scenario view. Session data such as transcripts and scores will also be stored in the database after the session is completed and the user's conversation is scored. User data will be accessed during authentication, session management, and when accessing conversation history.

6.4.3 Database Security Section

This database uses row level security to determine who has access to what data. This ensures that only authenticated users can access their own information, and no other user can. User data is stored securely in the database and protected from any unauthorized access. If a user is not authenticated, they cannot access any information. This means a user can read and edit their own profile, but not anyone else. It is very simple to setup security rules through Supabase's built in tools. All communication between the application and the database is performed over a secure HTTPS connection.